·雷达系统与技术·

DOI: 10.16592/ j. cnki. 1004-7859. 2022. 11. 013

基于大数据的雷达数据快速存储与分析研究

肖立志

(河南开放大学 教师网络学院， 郑州 450008)

摘要: 针对从大量雷达数据中发现频繁序列的问题,构建了一个基于 Hadoop 和 Spark 的雷达数据存储和分析系统。通过 Hadoop 和 Spark 的结合,可实现大量雷达数据的快速存储和分析。在所提系统中,Hadoop 使用分布式方式提供稳定高效的数据存储,Spark 提供高效的内存计算,建立了一个三节点的 Hadoop-Spark 集群执行分布式数据挖掘算法。此外,为提高雷达数据的分析精度,对雷达数据进行预处理且对挖掘结果进行后处理。结果表明: 文中方法可高效准确地存储与分析海量的雷达数据。

关键词: 雷达数据; 大数据; Hadoop; 数据挖掘

中图分类号: TN957. 52 **文献标志码:** A **文章编号:** 1004-7859(2022) 11-0079-06

引用格式: 肖立志. 基于大数据的雷达数据快速存储与分析研究[J]. 现代雷达, 2022, 44(11): 79-84.

XIAO Lizhi. Rapid storage and analysis of radar data based on big data[J]. Modern Radar, 2022, 44(11): 79-84.

Rapid Storage and Analysis of Radar Data Based on Big Data

XIAO Lizhi

(E-learning Academy for Teachers, HeNan Open University, Zhengzhou 450008, China)

Abstract: A radar data storage and analysis system based on Hadoop and Spark is constructed for the problem of discovering frequent sequences from large amounts of radar data. The combination of Hadoop and Spark enables the fast storage and analysis of large amounts of radar data. In the proposed system, Hadoop uses a distributed approach to provide stable and efficient data storage, and Spark provides efficient in-memory computing. A three-node Hadoop-Spark cluster is established to execute distributed data mining algorithms. In addition, to improve the accuracy of radar data analysis, the radar data is pre-processed and the mining results are post-processed. The experimental results show that the proposed method can store and analyse large amounts of radar data efficiently and accurately.

Key words: radar data; bigdata; Hadoop; data mining

0 引 言

雷达技术在现代战争中发挥着越来越重要的作用。雷达信号中的部分参数,如无线电频率(RF) 、到达方向(DOA) 、脉冲宽度(PW) 、到达时间(TOA) 、脉冲振幅(PA) 可代表雷达信号的特征,对于雷达信号的分类十分重要。雷达数据中的每个参数存在相近的模式,从雷达数据中发现特定的模式有助于雷达信号的分类。为此需要找到完整的频繁序列,分析处理海量的雷达数据。传统的单机处理方式无法实现高效的数据存储和挖掘,因而使用大数据的方法构造更加有效的存储和数据挖掘系统[1]。

近年来,随着大数据技术的发展,大数据已经被应用到各个领域,包括对雷达数据的处理和优化[2-3]。分布式计算框架的快速发展使得使用计算机集群处理数据变得更加高效。此外,Hadoop 和 Spark 的发展为雷达大数据的并行分析提供了基础[4-5]。Hadoop 主要由 Hadoop 分布式文件系统(HDFS) 和 MapReduce 构成。其中,HDFS 用于稳定地储存大量数据,MapReduce 用于高效并行计算。Spark 是另一种分布式计算框架,可与 Hadoop 协同工作,其与 MapReduce 的差异在于 MapReduce 主要在磁盘上处理数据,而 Spark 是在内存中计算,计算速度更快[6]。Hadoop 和 Spark 已被广泛用于多种大数据场景[7-8],近年来也被广泛用于雷达大数据的存储和分析[9-11]。

为了分析海量的雷达数据,文中提出了一个基于 Hadoop 和 Spark 的存储和分析系统,其创新点在于使用大数据技术实现了雷达数据的快速存储和数据挖掘,性能表现显著优于传统分析系统。系统首先对雷达数据进行预处理,包括去噪、符号化和数据分割。然后,使用 HDFS 存储大量的雷达数据,用 Spark 实现高效的并行计算。文中整合了 Hadoop 和 Spark 的优势以实现系统效率最大化。此外,文中还采用了 Spark 机器学习库(MLlib) 中的序列模式挖掘算法 PrefixSpan,以分布式方式挖掘频繁序列。最后,对挖掘结果进行了后处理。文中构建了一个三节点的 Hadoop-Spark 集群实现分布式数据挖掘计算,分布式 PrefixSpan 算法的挖掘性能显著高于单节点PrefixSpan算法。结果表明: 存储与分析系

收稿日期: 2022-08-05 修订日期: 2022-09-09

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net

统可实现对海量雷达数据的高效存储和数据挖掘。

1　雷达信号特性与大数据技术分析

1.1　雷达信号特性分析

雷达可向某一方向发射电磁脉冲,通过测量收到的信号和发射的信号之间的差异来确定目标物体的距离、速度、形状等。因此,雷达信号的发送是有规律的,以便从中提取所需信息。为此,雷达数据中存在特定的模式。例如,脉冲雷达由脉冲波组成,不同脉冲组合可形成多种信号模式。随着无线电电子技术的快速发展,雷达在现代战争中发挥着越来越重要的作用。雷达信号的分类对于雷达数据分析尤为重要,其中TOA、RF、DOA、PW 和 PA 是雷达信号分类的五个重要参数,具体描述如下:

(1) TOA: 接收器搜索和拦截脉冲的时间。

(2) PF: 脉冲的载波频率。RF 参数的调制形式多样,主要是为了解决模糊和抗干扰的问题。

(3) DOA: 辐射源和雷达接收器之间的相对方位角。该参数的值与接收器和目标辐射源的相对位置有关。

(4) PW: 由雷达本身产生的瞬时参数,通常被定义为脉冲半功率点之间的时间。

(5) PA: 主要与信号功率有关,会影响其他参数的测量精度。信号分类的可靠性可以通过 PA 的大小来描述。

雷达信号的脉冲描述字(PDW)可用式(1)表示为

$$PDW = \{TOA, PF, DOA, PW, PA\} \tag{1}$$

雷达数据中的每个参数都存在特定模式,对雷达数据进行分析时,需要分析每个参数的数据,并根据数据挖掘得到的每个参数的模式做出决策[12]。

1.2　Hadoop 特性分析

Hadoop 是一个开源软件框架,可处理大数据集的分布式应用,可基于简单的编程模型实现大规模数据的集群处理。其具备高效性、可靠性和应用的可扩展性等特性。Hadoop 主要由 HDFS 和 MapReduce 这两个组件组成[13]。

HDFS 可用于存储大量的数据。HDFS 容错性较强,且可以部署在较为廉价的硬件上,其性能超过了现有的分布式文件系统。HDFS 采用一个主从架构。一个 HDFS 集群由一个 NameNode 和多个 DataNode 组成。NameNode 作为主服务器来管理文件系统的命名空间,并调节客户端对文件的访问。文件存储的基本单位是 DataNode,它可以在本地文件系统中存储文件块,保存文件块的原数据,并定期向 NameNode 发送所有现有文件块的报告。一个文件被分割成一个或几个块,这些块被存储在一个 DataNode 中。命名空间的操作由 NameNode 执行,包括打开、关闭和重命名目录。DataNode 需要处理客户端发送的读写请求。此外,DataNode 将根据 NameNode 的指令删除、创建和复制块。HDFS 的架构如图 1 所示。

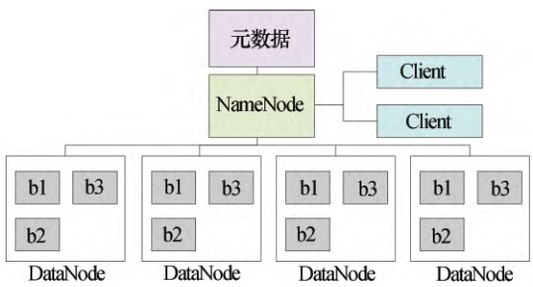

图 1　HDFS 架构图

MapReduce 是由谷歌提出的用于大规模数据集的并行计算框架[14]。MapReduce 基于“分而治之”的思想,通过 Map 和 Reduce 两个步骤实现数据的并行计算。首先,将输入的数据按照计算节点分割成多个数据块,然后数据块被分配给 Map 任务。每个 Map 任务将键值对作为输入,Map 的输出以键值的方式生成中间值,并根据键值进行排序,然后交付给 Reduce 任务。多个 Reduce 任务的 MapReduce 数据流如图 2 所示。

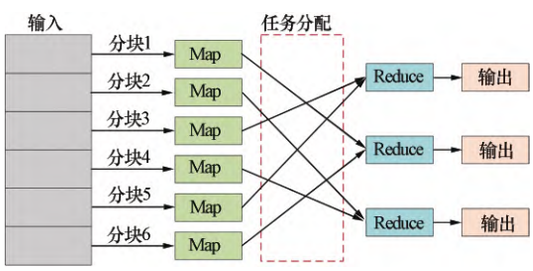

图 2　MapReduce 数据流图

尽管 MapReduce 已经能够实现较为高效的并行计算,然而依旧存在一些问题,如密集使用磁盘、内存计算不平等、迭代计算的能力较低、内部通信性能差等。MapReduce 的上述限制也促使了 Spark 的出现和发展。

1.3　Spark 特性分析

Spark 是一个大数据处理的开源框架,是 Hadoop 生态系统的重要组成部分,可与 Hadoop 联合使用。Spark 提供内存计算,计算速度相较于在磁盘上运算的 MapReduce 更快。Spark 基于弹性分布式数据集(RDDs),通过对 RDDs 的操作实现任务的分配。此外,RDDs 允许程序员将中间结果保存在内存或磁盘中以便重复使用,并为数据放置优化定制分区。除了 Spark Core 之外,Spark 还包含一些核心功能[15],具体介绍如下:

(1) MLlib: 由常见的学习算法和统计工具构成,包括分类、回归、聚类、协同翻译、降维和优化等,可简化大规模环境中的机器学习设计。

(2) Spark SQL: 引入了 DataFrame 此类用于半结

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net

构化和结构化数据的数据结构,支持在 Spark 程序中使用 SQL 查询。

(3) Spark 流:支持在流环境中快速处理小批量数据,使得同一套批处理代码可以直接用于流式分析。

(4) Graph X:Spark 中的一个图形处理项目。支持查看、转换和加入连接图和集合中。

由于 Spark 相对于 MapReduce 优势巨大,文中提出的系统,将 Hadoop 和 Spark 进行结合以最大限度地提高系统的效率,使用 Hadoop 中的 HDFS 实现大数据的高效存储,并使用 Spark 进行分布式计算。

2　系统架构

本文提出的系统的架构如图 3 所示,包含预处理层、数据挖掘层和后处理层三层结构。图 4 为该系统的工作流程。首先,对原始雷达数据进行去噪处理;其次,对连续数据进行离散化处理,并将数据分割成若干段,以便进行数据挖掘,完成预处理后,数据被存储在 HDFS 中;然后,应用 Spark MLlib 提供的挖掘算法 PrefixSpan 对数据进行挖掘;最后,对挖掘结果进行后处理工作。

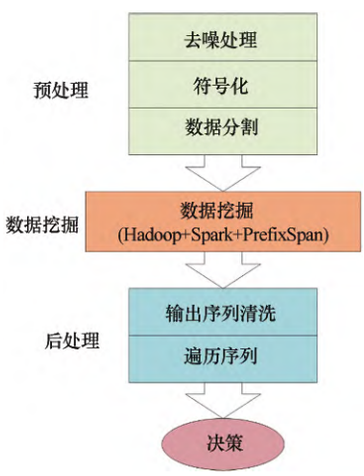

图 3　本文系统结构

图 4　文中系统工作流程图

2.1　雷达信号预处理

在文中系统中,雷达信号的预处理是一项重要的工作,可通过提高数据质量来提高挖掘结果的精度。预处理过程将原始雷达信号转换成适合后续处理的格式,包括去噪、符号化和数据分割这三个主要步骤。

2.1.1　去噪处理

有效的数据通常存在特定的有效范围,超出了有效范围的数据点可认为是噪声点而剔除。因此,首先可以基于先验知识删除一些数值明显不符合逻辑的数据。然后,可以使用基于密度的聚类算法(DBSCAN)对数据进行进一步去噪。DBSCAN 是一种典型的基于密度的聚类算法,常用于数据去噪处理。与 k-means 等聚类算法相比,DBSCAN 无需设定聚类个数,且可发现任意形状的聚类簇,可将明显离群的数据作为噪声进行去除。算法参数包括半径 R 和密度阈值 $MinPts$,算法的具体实施过程描述如下:

(1) 给定数据库 D_1 中的每个数据点都有两个标记:是否被访问和是否是噪音点。D_1 中的所有数据点都初始化为未访问点和噪声点。

(2) 从 D_1 中随机选择一个被标记为未访问的数据点 i。找到所有满足添加 $|i-x|<R$ 的数据点 x 组成的集合 X。

(3) 如果 X 中的数据点的数量大于或等于 $MinPts$,X 中的所有点都被标记为被访问的非噪声点;否则,不进行任何操作。

(4) 重复步骤(2)和(3),直到 D_1 中的所有对象都被标记为被访问的对象。删除 D_1 中仍被标记为噪音点的对象。

估计 R 的公式为

$$R=\frac{(\max(D_1)-\min(D_1))\times R\times\Gamma(1.5)}{\mathrm{len}(D_1)\times\sqrt{\pi}} \quad (2)$$

式中:$\max(D_1)$ 和 $\min(D_1)$ 分布表示 D_1 中的最大值和最小值;$\mathrm{len}(D_1)$ 表示 D_1 中的元素数量。

2.1.2　符号化

符号化类似于数字信号处理中的量化,是一种对连续数据进行分类的方法,使用统一的符号标记同一类别的所有数据,从而实现连续数据的离散化。与量化不同的是,给数据加上符号只是为了将其与其他类的数据区分开来,因而符号是可选的。由于 PrefixSpan 只能处理离散数据,所以在数据挖掘之前应该进行符号化。

为了与数据挖掘算法兼容,文中使用从 1 开始递增的正整数作为不同类别的符号。去噪后的数据被投射到 y 轴上,然后用 DBSCAN 对 y 轴上的数据进行聚类。一个类中的所有数据点都有一个相同的符号。

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net

DBSCAN 算法将数据点聚成四个类，一个类中的所有数据点都有一个相同的符号，如 1、2、3、4。符号化后的数据点具有值和符号这两个标签。符号化公式为

$$\begin{cases} C(c_1,c_2,\cdots,c_n) = \mathrm{DBSCAN}(a_1,a_2,\cdots,a_m),n < m \\ S(s_1,s_2,\cdots,s_n) = \mathrm{Tag}(c_1,c_2,\cdots,c_n) \end{cases} \tag{3}$$

式中：$C(c_1,c_2,\cdots,c_n)$ 表示聚类结果；$S(s_1,s_2,\cdots,s_n)$ 表示符号化的结果。

2.1.3　数据分割

数据挖掘技术一般处理多段数据，而收集的雷达数据只有一段。因此，需要将数据分割成若干段。文中的数据分割方法遵循两种策略：去噪后的数据点的原始位置被认为是分段点；每段的长度不能大于 30。采用的去噪方法是直接去除噪声点而不填充其他数据点，所以去除噪声点两边的数据在时间上是不连续的。如果直接进行数据挖掘将会得到错误的结果。因此，所有去噪点的原始位置被用作分段点，以确保每个分段的数据在时间上是连续的。此外，根据先验知识，雷达信号模式的长度通常不大于 30，所以每个段的最大长度被设定为 30。

在整个分割过程中，数据的顺序始终保持不变，基于上述策略，分割的过程如下：

(1) 符号化的数据以所有的噪声点作为分段点进行划分。

(2) 将步骤(1)中长度大于 30 的片段分割成长度等于或小于 30 的若干片段。

(3) 储存分段结果，每个分段为一个数据挖掘的序列。

2.2　数据挖掘

系统使用 Spark MLlib 中的 PrefxSpan 算法进行数据挖掘，并且使用了分布式的方式实现雷达数据挖掘。

2.2.1　序列模式挖掘

顺序模式挖掘是指找出关于给定数据库 D_2 和阈值 min _ sup 的频繁序列。假定 D_2 为数据库，其中每个交易包含客户 ID 和一个数据项。数据项为 $I = \{i_1, i_2, \ldots, i_m\}$。序列 $S = <s_1, s_2, \ldots, s_n>$为按字母顺序排列的数据项集合列表，一个包含 k 个数据项的序列被称为k序列。例如，$<(1)(2)(3)(4)>$，$<(2)(4)(1)(3)>$和$<(1)(3)(2)(4)>$都是 4 序列。序列 $\alpha = <\alpha_1, \alpha_2, \cdots, \alpha_n>$可被视为序列 $\beta = <\beta_1, \beta_2, \cdots, \beta_m>(n \leqslant m)$ 的子序列，如果存在整数 $1 \leqslant i_1 < \cdots < i_n \leqslant m$，则$\alpha_1 \subseteq \beta_{i1}, \cdots, \alpha_n \subseteq \beta_{in}$。例如，$<(1)(2)>$是$<(2)(1)(3)(2,4)>$的子序列，因为$(1) \subseteq (1)$，$(2) \subseteq (2,4)$。序列 S 的支持度是数据库中出现的 S 的数量，表示为 $\sup(S)$。如果 $\sup(S) \geqslant \min_\sup$，那么序列 S 就被称为序列模式。一个长度为 1 的序列模式可以被称为 1-模式。如果 S 不存在子序列，那么 S 可以被称为数据库中的最大序列模式。

2.2.2　PrefxSpan 算法

PrefxSpan 是一种基于递归构建模式的挖掘算法。在 Spark MLlib 中使用 PrefxSpan 算法来寻找雷达数据中的频繁序列。其计算步骤如下：

(1) 遍历投影数据库 $S|\alpha$，找到一组连续的项 b。

(2) 将每一个连续的项 b 添加到 α 上形成一个新的序列 α'。

(3) 针对每一个 α'创建一个投影数据库 $S|\alpha'$，并调用 $\mathrm{PrefixSpan}(\alpha', l+1, S|\alpha')$。

2.2.3　Spark 中的挖掘过程

文中建立了一个三节点的 Hadoop-Spark 集群用于分布式计算。在一个任务的执行过程中，有一个应用主程序和几个执行程序。具体过程如图 5 所示。步骤如下：

(1) 将存储在 HDFS 中的数据转换为 RDD；

(2) 扫描数据库生成频繁项目以及每个项目的大型投影数据库。

(3) 由于投影数据库很大，在大的投影数据库中寻找频繁序列的任务被分配给几个执行者来处理，直到投影数据库变小。在这个过程中产生的频繁序列被存储在应用主程序上。依次对每个大型投影数据库进行同样的处理。

(4) 每个小的投影数据库都由本地的执行者处理，以分布式的方式生成频繁序列。

(5) 生成的序列进行汇聚并转换为数组格式，作为最终的挖掘结果。

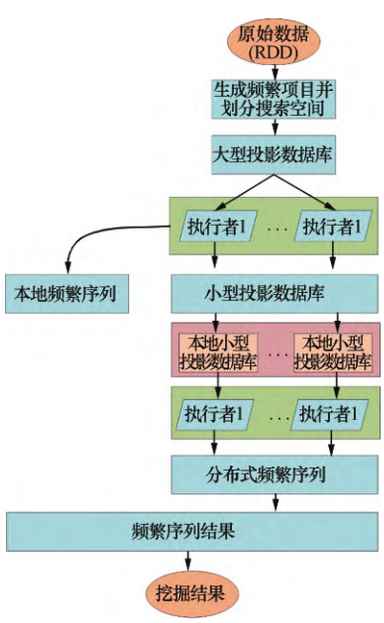

图 5　挖掘流程图

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net

2.3　数据后处理

通过数据挖掘可以得到雷达数据中完整的频繁序列。然而数据挖掘的结果存在一些不足之处:(1) 挖掘结果的数量较大,例如,从 100 个数据点中可能挖出 1 000多个序列;(2) 结果的冗余度很高,例如,如果<(1)(2)(3)>是挖掘结果中的一个序列模式,<(1)(2)>作为子序列应该作为冗余删除;(3) 可能存在时间上不连续的结果。为了解决上述问题,文中使用了后处理的方法,通过只保留最大的序列模式来优化结果。首先,对输出序列进行清洗,去除大量序列;然后,对保留的序列进行遍历,寻找最大的序列模式;最后,通过执行后处理方法,有效去除挖掘结果中的冗余部分。

2.3.1　输出清洗

通过对挖掘序列进行清洗,有助于减少后续的遍历结果的工作量。对挖掘结果的清洗思路如下:

首先,删除所有 $k<3$ 的序列。1-序列不能反映按字母顺序排列的数据之间的规律性,而 2-序列由于其长度较短,存在大量的随机性。因此,$k<3$ 的序列应该删除。

然后,对剩余序列的长度进行分析,得到最大长度的序列模式。由于 PrefixSpan 算法采取的是深度搜索策略,并不限制挖掘间隙,所以具有包容关系的序列模式会依次出现,根据这一分布特征,可以找到所有序列长度的一阶差分,计算方式如式(4) 所示,并删除为 0 或 1 的序列,对剩余序列进行多次翻转后重复这一过程,最终得到长度最长的序列模式。

$$L(i) = \mathrm{len}(S_{i+1}) - \mathrm{len}(S_i) \tag{4}$$

式中:$L(i)$ 表示输出结果中第 i 个序列的一阶差分结果;$\mathrm{len}(S_{i+1})$ 表示第 $i+1$ 个序列的长度;$\mathrm{len}(S_i)$ 表示第 i 个序列的长度。

2.3.2　遍历剩余序列

经过清洗后,挖掘结果中仍然存在冗余序列。使用遍历保留序列的方法来进行最终的筛选,具体过程如下:

(1) 按长度升序对所有序列进行排序。

(2) 遍历所有序列,将每个序列长度与结果中所有其他序列进行对比。

(3) 一旦发现当前序列是任何其他序列的子序列,则删除当前序列。

(4) 重复上述步骤直到所有的序列都被遍历。最终保留的序列即为最大序列模式,将其作为挖掘结果。

3　实验与结果分析

文中实验系统在三台电脑上使用 Hadoop 和 Spark 框架搭建而成。操作系统为 Ubuntu 16.04。通过实验验证了本文系统能够快速、准确地从雷达数据中挖掘出频繁的序列。从两个方面评估了系统的整体性能,即分析系统的准确性和 Hadoop-Spark 集群的性能。为了测试分析系统的准确率,文中使用了三种参数的雷达数据,雷达数据来自于机载雷达对地面扫描的数据,数据包含 TOA、RF、DOA、PW 和 PA 等参数,并进行结构化处理,便于存储和分析,数据量分别为500 MB、1 GB 和 2 GB,min_sup 参数设为 0.01。实验结果如表 1 所示。

表 1　雷达数据序列模式挖掘结果

数据量	发现的序列数	模式准确率/%	支持度准确率/%	运行时间/min
500 MB	2 085	100	85	13
1 GB	4 130	100	86	23
2 GB	8 069	100	86	45

由表1可知,在数据量为500 MB时,一共发现 2 085 个序列,此时模式的准确率为 100%,而支持数的准确率由于噪声的影响仅为 85%,运行时间为 13 min。数据量为 1 GB 和 2 GB 时,挖掘结果同样较为准确率,模式准确率均为 100%,支持度准确率为 86%,运行时间分别为 23 min 和 45 min。结果表明:文中系统能够实现对雷达数据模式序列的准确分析挖掘。

为了验证文中所使用的 Hadoop-Spark 集群处理的性能,首先,对比单机 Hadoop-Spark 系统与传统单机的处理性能;然后,将单机 Hadoop-Spark 系统与 Hadoop-Spark 集群系统的处理性能进行了对比,结果分布如图 6 和图 7 所示。

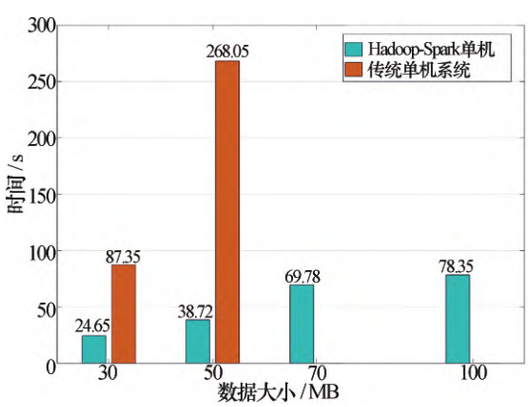

图 6　Hadoop-Spark 与传统单机系统处理性能对比

图 6 为 PrefixSpan 算法在传统单机和 Hadoop-Spark 单机中的数据挖掘性能对比。如图 6 所示,当采用 30 MB 和 50 MB 的数据集时,Hadoop-Spark 的计算时间显著少于传统单机,且在数据超过 70 MB 时,传统单机已经出现错误无法处理。对比实验结果表明:传统的单机无法处理大型数据集,而基于 Hadoop 和 Spark 的系统不仅能够支持对大量数据的处理,处理速

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved.　http://www.cnki.net

度也显著提升。图7中进一步比较了Hadoop-Spark单机与集群的处理性能对比,对比实验表明:Hadoop-Spark集群的计算速度显著优于单机,且随着数据量的增大,性能优势更加明显。结果表明:文中系统支持对大数据的高效存储和数据挖掘。

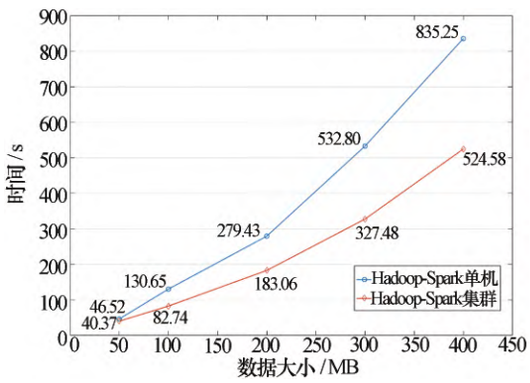

图7 Hadoop-Spark单机与集群处理性能对比

4 结束语

为了实现对雷达大数据的高效存储和数据挖掘,提出了一种基于Hadoop和Spark的雷达数据分析系统。该系统基于Hadoop设计了高效的雷达数据存储模块,使用Spark设计了高性能的并行数据挖掘模块,并且建立了多个节点的集群来实现更高的数据挖掘性能。为了提高雷达数据的分析精度,提出了对雷达数据的预处理以及对挖掘结果的后处理方法。实验结果表明:文中方法可实现对雷达数据的有效挖掘,同时计算速度显著优于对比方法。

参考文献

[1] TONG Z, GAO J, YUAN D D. Advances of deep learning applications in ground-penetrating radar: a survey [J]. Construction and Building Materials, 2020, 258: 120371-1-120371-14.

[2] 申 娟,宋正鑫,郭建明,等. 大数据在多功能雷达性能阵地现场优化中的应用[J]. 现代雷达,2019,41(11):10-13.
SHEN Juan, SONG Zhengxin, GUO Jianming, et al. Application of big data in site performance optimization for multi-function radar [J]. Modern Radar, 2019, 41(11): 10-13.

[3] 杜 力. 基于大数据的船舶雷达信息存储优化方案研究[J]. 舰船科学技术,2019,41(6):116-118.
DU Li. Research on optimal storage scheme of ship radar information based on large data [J]. Ship Science and Technology, 2019, 41(6): 116-118.

[4] MOSTAFAEIPOUR A, RAFSANJANI A J, AHMADI M, et al. Investigating the performance of Hadoop and Spark platforms on machine learning algorithms [J]. The Journal of Supercomputing, 2021(2): 1-28.

[5] YANG C, YANG X M, YANG F. A system based on Hadoop for radar data analysis [J]. Journal of Ambient Intelligence and Humanized Computing, 2019, 10(10): 3899-3913.

[6] SHI J W, QIU Y J, MINHAS U F, et al. Clash of the titans: MapReduce vs. Spark for large scale data analytics [J]. Proceedings of the VLDB Endowment, 2015, 8(13): 2110-2121.

[7] XU X. Safe storage method of ship communication data based on double key and Hadoop [J]. Journal of Coastal Research, 2020, 103(S1): 961-965.

[8] CHEN D H, ZHANG R T. Catla-HS: an open source project for tuning and analyzing MapReduce performance on Hadoop and Spark [J]. IEEE Software, 2020, 39(1): 61-69.

[9] DEIBE D, AMOR M, DOALLO R. Big data storage technologies: a case study for web-based LiDAR visualization [C]// 2018 IEEE International Conference on Big Data. Seattle: IEEE Press, 2018: 3831-3840.

[10] WANG G Q, CHEN H, XIE Y Z. An efficient dual-channel data storage and access method for spaceborne synthetic aperture radar real-time processing [J]. Electronics, 2021, 10(6): 662-1-662-22.

[11] KNEPPER R, STANDISH M. Forward observer system for radar data workflows: big data management in the field-ScienceDirect [J]. Future Generation Computer Systems, 2017, 76: 92-97.

[12] 谭怀英,王丁禾,郭建明,等. 大数据挖掘技术在预警探测信息处理领域的应用前景[J]. 现代雷达,42(11):1-5.
TAN Huaiying, WANG Dinghe, GUO Jianming, et al. Application prospect of big data mining technology in early warning detection information processing [J]. Modern Radar, 42(11): 1-5.

[13] GHAZI M R, GANGODKAR D. Hadoop, MapReduce and HDFS: a developers perspective [J]. Procedia Computer Science, 2015, 48: 45-50.

[14] DEAN J, GHEMAWAT S. MapReduce: a flexible data processing tool [J]. Communications of ACM, 2010, 53(1): 72-77.

[15] GARCÍA-GIL D, RAMÍREZ-GALLEGO S, GARCÍA S, et al. A comparison on scalability for batch big data processing on Apache Spark and Apache Flink [J]. Big Data Analytics, 2017, 2(1): 1-11.

作者简介:

肖立志 男,1980年生,硕士,副教授,研究方向为信息化、大数据。

(C)1994-2023 China Academic Journal Electronic Publishing House. All rights reserved. http://www.cnki.net